

CONTENT

ATTACHMENTS

Rule Syntax

8 months ago

Rule SyntaxRule Syntax

This section describes the different rule syntax supported in the Reporting Engine.

NWDB Rule SyntaxNWDB Rule Syntax

The NWDB rule is one of the rule syntax supported in the Reporting Engine. To enhance the execution time of your reporting entities, see "Reporting Guidelines" section in [Reporting Overview \(https://community.netwitness.com/s/article/ReportingOverview\)](https://community.netwitness.com/s/article/ReportingOverview).

A Rule is a function that manipulates the result set of a rule in order to make the output in a report more meaningful or add additional functionality to a rule other than querying data and displaying it. Any combination of these rule actions can be used to create unique and interesting representations of the information collected by NetWitness.

The Reporting Engine supports the following categories of NWDB data source rule syntax:

- **select** clause
 - Non-Aggregate Rule
 - Aggregate Rule
- **alias**
- **where** clause
- **where** clause Operators
- **then** clause
- **Limit** field
- Rule Actions
- Rule Operators

Select Clause

The select clause is a comma separated list of values. For example: select sessionid,time,service.

There are two types of select clause for NWDB Rule:

- Non-aggregate rule
- Aggregate rule

Non-Aggregate Rule

When you want to define a rule without any grouping, choose 'None' in the Summarize field. In a non-aggregate rule, you can select any number of metas in the *Select* clause. For example, select service, sessionid, time.

Build Rule

Rule Type

NetWitness Platform DB

Name

Non-Aggregate Rule

Summarize

None ▾

Select

service, sessionid, time

Alias

IP Source

Where

service=443

Then

Enter a then clause...

Order By

Column Name	Sort By
Enter the column name...	Ascending

Limit

20 ▾

Use

Save

Reset

Test Rule

Aggregate Rule

When you want to query for a specific meta and its associated aggregate value then you must use the Aggregate rule. To get an aggregate, you must choose either of the three metas (Event Count, Packet Count, Session Size) or choose 'Custom' in the **Summarize** field to include an aggregate function in the *Select* clause. For example, select ip.src, sum (ip.dst). When Custom aggregate rule is enabled, the following fields are populated in the user interface:

- Group By
- Order By
- Session Threshold

The following figure shows the Build Rule view for Aggregate Rule.

Build Rule

Rule Type: NetWitness Platform DB

Name: Aggregate Rule

Summarize: Custom

Select: ip.src, countdistinct(ip.dst)

Alias: IP Source

Where: ip.src exists

Group By: ip.src

Then: Enter a then clause...

Order By:

Column Name	Sort By
ip.src	Ascending
countdistinct(ip.dst)	Ascending
Enter the column name...	Ascending

Session Threshold: 0

Limit: 20

Buttons: Use, Save, Reset, Test Rule

There are two types of aggregate values that can be queried:

- Collection aggregation
- Meta aggregation

Collection Aggregation

With collection aggregation, you can get aggregates related to Event, Session or Packets. The following values can be queried in a collection aggregation:

- **Event Count:** The total count of events.
- **Packet Count:** The total count of packets.
- **Session Size:** The total session size.

These options are listed in 'Summarize' field and any one of them can be selected in a rule.

For example, choose any of the Collection aggregates (Event Count or Packet Count or Session Size) in the 'Summarize' field and select ip.src.

Build Rule

Rule Type

NetWitness Platform DB

Name

Collection Aggregation

Summarize

Event Count

Select

ip.src

Alias

IP Source

Where

Group By

ip.src

Then

Enter a then clause...

Order By

Column Name	Sort By
Total	Ascending

Session Threshold

0

Limit

20

Use

Save

Reset

Test Rule

Meta aggregation

With meta aggregation, you can get aggregates of meta values. The following are the supported meta aggregate functions:

- sum(meta)
- count(meta)
- countdistinct(meta)
- min(meta)
- max(meta)
- avg(meta)
- first(meta)
- last(meta)
- len(meta)
- distinct(meta)

Supported Meta Aggregate Functions

The NWDB service supports the following meta aggregate functions and syntax in this release.

Syntax	Function
sum(<meta>)	The sum of all meta values. For example, if you provide the field sum(payload) in the select clause, the resultset is the sum of payload size. Note: The meta field chosen for the sum aggregate function must be of numeric data type.
count(<meta>)	The total number of meta fields that would be returned. For example, if you provide the field count(ip.dst) in the select clause, the resultset is the number of times an ip.dst value is returned.
countdistinct(<meta>)	The total number of distinct meta fields that would be returned. For example, if you provide the field countdistinct(ip.dst) in the select clause, the resultset is the number of times a distinct ip.dst value is returned.
min(<meta>)	The minimum of all meta values. For example, if you provide the field min(payload) in the select clause, the resultset is the min of payload size.
max(<meta>)	The maximum of all meta values. For example, if you provide the field max(payload) in the select clause, the resultset is the max of payload size.

Syntax	Function
avg(<meta>)	The average of all meta values. For example, if you provide the field avg(payload) in the select clause, the resultset is the avg of payload size. Note: The meta field chosen for the avg aggregate function must be of numeric data type.
first(<meta>)	The first occurrence of the meta value. For example, if you provide the field first(ip.src) in the select clause, the resultset is the first occurrence of ip.src for that group.
last(<meta>)	The last occurrence of the meta value. For example, if you provide the field last(ip.src) in the select clause, the resultset is the last occurrence of ip.src for that group.
len(<meta>)	Converts all field values to a UInt32 length instead of returning the actual value. This length is the number of bytes to store the actual value, not the length of the structure stored in the meta database. For instance, the meta value "NetWitness" returns a length of 10. All IPv4 fields, like ip.src, returns 4 bytes.
distinct(<meta>)	The distinct values of the meta. For example, if you provide the field distinct(ip.src) in the select clause, the resultset is all the distinct ip.src for that group.

You must select 'Custom' in 'Summarize' field and provide the meta and the meta aggregate functions in the select clause.

Build Rule

Rule Type:

Name:

Summarize:

Select:

Alias:

Where:

Group By:

Then:

Order By:

Column Name	Sort By
ip.src	Ascending
Enter the column name...	Ascending

Session Threshold:

Limit:

Buttons:

Note: Meta aggregate functions cannot be used in a WHERE clause and the rule actions like min_threshold/max_threshold can be used to filter aggregate functions. It is advised to use a more refined WHERE clause to get a better rule performance while using 'group by'.

Aggregate Query for Multiple Meta

To execute aggregate query for multiple Meta, follow these steps:

1. Go to **Reports**.
The Manage tab is highlighted and the **Rules** view is displayed.
2. In the **Rules** toolbar, click **+** > **NetWitness Platform DB**.

For example, enter the following meta in the fields highlighted below:

SELECT: ip.src, service, count(alias.host)

ALIAS: Source IP Address, Service Type, count(alias.host)

WHERE: ip.src = 59.96.136.142

Note: In the alias field you can enter a name for columns used in the select clause. If you do not specify the alias for one of the field in the select clause, then the default description will be used. For example, if the select clause has Field1, Field2, Field3, Field4, and alias has only Field1, Field3, Field4, then for Field2 a default description is used.

3. Click the **Test Rule** button at the bottom of the screen.

The Test Rule page is displayed.

Summarize

Summarize determines the type of summarization or aggregation for the rule.

Name	Config Value
------	--------------

To query metas without any custom grouping, select:

- **None:** The data is grouped by session in this case.

To get collection (sessions/events/packets) related aggregates, select either of the following:

Summarize

- **EventCount:** The total count of events.
- **Packet Count:** The total count of packets.
- **Session size:** The total session size.

To get meta based aggregates, select:

- **Custom:** This indicates that expected meta aggregate function is defined in rule select clause.

Order By

Order By determines how to sort the result set.

Name	Configuration Value
------	---------------------

The **Column Name** is the name of the columns by which you want to sort the results. By default, the value is empty. When you click on a column, the value gets populated based on the Summarize field.

Column Name

- For 'None' and 'Custom', the value gets populated based on the entries made in the Select field. You can select from this list or add custom name.
- For Event Count, Packet Count and Session size, accepted values are Total and Value.
- Total - sort by aggregate value
- Value - sort by group by meta

Sort By determines the order in which you want to sort the results. The following are the values:

Sort By

- Ascending Order
- Descending Order

Session Threshold

The session threshold is the optimization setting to stop scanning the matching sessions for each possible unique value for the selected meta. The threshold is an integer between 0 (default) and 2147483647. The threshold 0 scans for all matching sessions.

Note: If you provide a non-zero value (a value higher than zero), the aggregate results are inaccurate. This can be used only when you are interested in unique values and not aggregate values.

Supported where Clause

Syntax	Description
where <field1> [<field-operator>] <value1>, <value2>, <value3>-<value4> <logic-operator> <field2>, and so on	The where clause is a comma separated list of language field values and ranges that is used by NwValues function. In the where clause, string values have to be enclosed within single quotes. For example, where username = 'admin' && service = 22.
where <field1> [<field-operator>] <List1>	You can use a list in the where clause if you have multiple values to report on. For example, where ip.src exists && alias.host exists && alias.host contains \$[User Reports/List of Alias Host]. When you use the list you must specify in the format \$[<path>/<List name>].

In the where clause, make sure the syntax is correct based on the meta type.

For example,

For all text meta type use quotes for example, username = 'user1'.

For all IP Addresses, Ethernet Addresses, and Numeric meta types do not use quotes for example, service = 80 && ip.src = 192.168.1.1.

For date and time meta types, if the date and time format is 'YYYY-MM-DD HH:MM:SS', use quotes.

If the date and time format is 1448034064 (number of seconds since EPOCH (Jan 1, 1970)), do not use quotes.

Note: If list is used in the rule, make sure that the list values are quoted or unquoted based on the type of the meta used. Checking the **Quotes will be inserted for all the values** checkbox in list definition page (for more information see, "Create Lists or List Groups" section in [Configure a Rule](https://community.netwitness.com/s/article/ConfigureaRule) (<https://community.netwitness.com/s/article/ConfigureaRule>)) would quote all the list values.

Supported where Clause Operators

Syntax	Description
=	Returns results where the field is equal to any provided value. For example, tcp.dstport = 21-25,110 returns session with TCP destination ports of 21, 22, 23, 24, 25, or 110.
!=	Returns results for fields that do not match the values specified. For example, eth.type != 0x0800 returns sessions outside of hex value (decimal value of 2048) that is all non-IP based protocols.
begins	Checks for a value at the beginning of a text or binary field.
contains	Searches a text or binary value for a partial match.
ends	Checks for a value at the end of a text or binary field.
exists	If the field value exists, regardless of value, the operation evaluates to true.
!exists	If the field value does not exist, the operation evaluates to true.
length	Evaluates the length of the field. For example, username length 20-u returns any username that is 20 or more characters long.
regex	Performs a regular expression search against text or binary values.
not	Not operator is used to negate a clause or condition. For example, (not(user.dst ends "\$")) will not display values for user destination.

Supported then Clause

Syntax	Description
then <rule action>	The then clause contains a rule action that manipulates the original result set of a rule in order to make the output in a report more concrete or add additional functionality other than querying data and displaying it. For example, dedup (filename).

Limit field

This indicates the limit to be put on the query while fetching data from the database. If a result set is sorted by event count, packet count, or session size, the limit represents the top (or bottom) N values to be returned. If the result set is not sorted, the first N values are returned.

Rule Actions

The NWDB data source rule syntax supports the following rule actions:

- dedup
- filter_on
- filter_out
- lookup_and_add
- max_threshold
- min_threshold
- regex
- sum_count
- sum_values
- show_whats_new

dedup (string field)

dedup removes the duplicate entries in an unsorted result set and displays only pertinent data. The dedup rule action removes duplicate entries of a specific field in the report, so that only the first occurrence of that value is listed in the report.

Note: The dedup rule action cannot be used with an aggregate rule.

For example, the meta data generated by an individual session is often repetitive, especially when you have sessions with a lot of DNS lookups or web sessions that access the same host multiple times for various resources (such as, javascript, css). To remove the duplicate entries of the host, you can use the dedup rule action.

Example:

The following example is a lengthy result set that can be trimmed by removing the duplicate values in the same session.

2015 01 27 04:05 Rule without Dedup Rule Actions				2015 02 10 04:05			
	Source IP Address	Service Type	Hostname Aliases				
1	192.168.1.100	SSL	Microsoft Secure Server Authority				
2	192.168.1.100	HTTP	thumbs3.ebaystatic.com thumbs3.ebaystatic.com				
3	192.168.1.100	HTTP	au.download.windowsupdate.com, au.download.windowsupdate.com, au.download.windowsupdate.com, au.download.windowsupdate.com, au.download.windowsupdate.com, au.download.windowsupdate.com				
4	192.168.1.100	HTTP	blackboard.jason.org				
5	192.168.1.100	HTTP	blackboard.gwu.edu				
6	192.168.1.100	HTTP	mail.google.com mail.google.com mail.google.com mail.google.com				
7	192.168.1.100	HTTP	gwired.gwu.edu				
8	192.168.1.100	HTTP	ads1.msn.com				
9	192.168.1.100	HTTP	www.skysports.com, www.skysports.com, www.skysports.com, www.skysports.com				
10	192.168.1.100	HTTP	server.cpmstar.com				
11	192.168.1.100	HTTP	www.gwu.edu, www.gwu.edu				
12	192.168.1.100	HTTP	pf1.imag.gwu.edu, pf1.imag.gwu.edu, pf1.imag.gwu.edu,				

The following figure shows the use of dedup rule action to remove the duplicate entries from the result set.

Build Rule

NetWitness Platform DB

Name: Rules with Dedup Rule Actions

Summarize: None

Select: ip.src, service, alias.host

Alias: Source IP Address, Service Type, count(alias.host)

Where: ip.src exists && service.exists && alias.host exists

Then: dedup(alias.host);
Enter a then clause...

Order By: Column Name: Enter the column name... Sort By: Ascending

Limit: 1000

Use Save Reset Test Rule

The duplicate value for each entry in the rule result set is reduced to one value.

Test Rule

Data Source: 204.31-Conc

Format: Tabular

Time Range: Past

2 Weeks

☒ Use relative time calculation

Run Test

	Source IP Address	Service Type	Hostname Aliases
1	198.146.75.100	SSL	Microsoft Secure Server Authority
2	191.206.161.100	HTTP	thumbs3.ebaystatic.com
3	191.206.16.107	HTTP	au.download.windowsupdate.com
4	191.206.126.7	HTTP	blackboard.jason.org
5	191.206.86.24	HTTP	blackboard.gwu.edu
6	191.206.6.8	HTTP	mail.google.com
7	198.146.152.22	HTTP	gwired.gwu.edu
8	191.206.6.207	HTTP	ads1.msn.com
9	191.206.24.8	HTTP	www.skysports.com
10	191.206.4.200	HTTP	server.cpmstar.com
11	88.174.145.200	HTTP	www.gwu.edu
12	27.96.243.145	DNS	pf1.imag.gwu.edu
13	88.174.145.200	HTTP	www.gwu.edu
14	198.146.27.10.200	HTTP	favicon.yandex.net

Close

filter_on (string filter, string field, bool matchExact)

filter_on removes values that do not contain the filter criteria from the result set. If the result set contains multiple fields, you must select a specific field to which the filter is applied. To add additional results to a single result set, include function such as lookup_and_add.

The matchExact parameter determines if the match is an exact match or contains a match.

- If matchExact is set to false, any value that contains the filter text is considered a match.
- If matchExact is set to true, only values that match the provided filter text is included in the result set.

Note: Unless the matchExact parameter is specified, the default behavior of the rule action is to match exactly the text specified in the filter parameter. To specify that results containing the filter text must be kept in the result set, users must set the matchExact parameter to false.

Example:

The following figure displays the list of countries and their event count.

Test Rule

Data Source: 204.31-Conc

Format: Tabular

Time Range: Range

From: 02/10/15 01:00:00

To: 02/10/15 03:00:00

Run Test

	Source Country	Total events count
1	united states	15105
2	china	1174
3	united kingdom	381
4	spain	362
5	canada	344
6	poland	318
7	france	285
8	germany	258
9	korea, republic of	203
10	brazil	200
11	italy	198
12	bulgaria	170
13	argentina	162
14	taiwan	160
15	iran	150

Close

The following figure shows a filter_on rule action to filter out countries except Spain, China, United States and United Kingdom from the result set.

Build Rule

Rule Type:

Name:

Summarize:

Select:

Alias:

Where:

Group By:

Then:

Order By:

Column Name	Sort By
Total	Descending

Session Threshold:

Limit:

The following figure shows the output with the filter_on rule action.

Test Rule

Data Source:

Format:

Time Range:

☒ Use relative time calculation

	2018 01 02 06:59:00	Rule with Filter_On_True	2018 03 02 06:58:59
	Country Source	Total events count	
1	china	329101	
2	spain	64649	
3	united kingdom	58389	

Another way of filtering out the entries from the result set is to create a list of variables which you want to filter out. For example, you can create a list with United Kingdom, France and Germany as values in the list. You can use this list in the rule action to get the same result set. For example, if you create a list called COUNTRY_LIST, you can use the list as follows:

```
filter_on ('$COUNTRY_LIST', 'country.src', 'false');
```

```
filter_out (string filter, string field)
```

```
filter_out (string filter, string field, bool matchExact)
```

filter_out removes the values that contain the *filter* criteria from the result set. If the result set contains multiple fields, you must select a specific field to which the filter is applied (for example, you can use a lookup_and_add to add results to a single result set).

The matchExact parameter determines if the match is an exact match or contains a match.

- If matchExact is set to false, any value that contains the filter text is considered a match.
- If matchExact is set to true, only values that match the provided filter text is excluded from the result set.

Note: Unless the matchExact parameter is specified, the default behavior of the rule action is to match exactly the text specified in the filter parameter. To specify that results containing the filter text must be removed from the result set, users must set the matchExact parameter to false.

Example:

The following figure displays the list of countries and their event count.

Test Rule

Data Source
204.31-Conc

Format
Tabular

Time Range
Range

From
02/10/15 01:00:00

To
02/10/15 03:00:00

Run Test

	2015 02 10 01:00	Rule without Filter_Out	2015 02 10 03:00
	Source Country	Total events count	
1	united states	15105	
2	china	1174	
3	united kingdom	381	
4	spain	362	
5	canada	344	
6	poland	318	
7	france	285	
8	germany	258	
9	korea, republic of	203	
10	brazil	200	
11	italy	198	
12	bulgaria	170	
13	argentina	162	
14	taiwan	160	
15	israel	150	

Close

The following figure shows the filter_out rule action to remove the event count for Spain, China, United States and United Kingdom from the result set.

Build Rule

Rule Type
NetWitness Platform DB

Name
Rule with Filter_Out_True

Summarize
Event Count

Select
country.src

Alias
Country Source

Where
country.src exists

Group By
country.src

Then
filter_on ('spain,china,united kingdom','country.src');

Order By

Column Name	Sort By
Total	Descending

Session Threshold
0

Limit
200

Use **Save** **Reset** **Test Rule**

The following figure shows the output with the filter_out rule action.

	Country Source	Total events count
1	china	329101
2	spain	64649
3	united kingdom	58389

lookup_and_add (string select, string field)

lookup_and_add (string select, string field, int limit)

lookup_and_add (string select, string field, int limit, boolean inherit)

lookup_and_add (string select, string field, int limit, boolean inherit, string extraWhere)

lookup_and_add(string select, string field, int limit, boolean inherit, string extraWhere, boolean aggregate)

This rule action iterates through a list of values in a result set and lookup additional meta data to further describe the relationships between various elements in a result set.

Note: The lookup_and_add rule action can be used only with an aggregate rule.

The first parameter, select, designates the type of meta data that must be added to elements of the result set. The second parameter, field, specifies where in the result set the append must apply to. Also, a limit can be applied to avoid crowding the result set with a large result set.

By default, subsequent queries to the SDK will inherit the where clause of the parent rule. To use a unique where clause, you can specify a boolean value in the fourth parameter as false and in the fifth parameter you can specify a different where clause.

Note: If you are using a unique where clause in your query, make sure that you use a single quote (') for enclosing arguments and double quotes (") for string values.

Now, with the addition of **Custom** summarization and **Group By** feature, the result can be achieved even without having lookup_and_add rule action. The new rule syntax with groupby displays the result in a flat structure which is better than the earlier rule syntax without groupby. Hence it is recommended to manually edit/update rules with lookup_and_add rule action and use groupby clause wherever it is applicable.

Note: Lookup_And_Add rule action is supported only if the SELECT clause has one meta and aggregate function.

For example, see below scenarios: In Example 2a, lookup_and_add rule action is used. Instead of using lookup_and_add rule action, the same result can be achieved by using **Custom** summarization and **Group By** feature. See Example 2b below.

But, lookup_and_add rule action is still supported for NWDB rules on the following conditions:

- All versions of NWDB rules with Summarization as Event Count, Packet Count, or Session Size.
- For Custom summarization, the lookup_and_add rule must have only one group by meta with only one aggregate function where the aggregate function must be either sum() or count().

Note: It is not supported for "Summarize-None".

For example, lookup_and_add rule action can be used for the following rules:

- select ip.src, sum(size) group by ip.src
- select ip.src, count(filename) group by ip.src

It cannot be used for the following rules:

- select ip.src, sum(size),count(filename) group by ip.src
- select ip.src, sum(size),avg(size) group by ip.src
- select ip.src,ip.dst count(filename) group by ip.src,ip.dst

Examples:

1. lookup_and_add('ip.dst','ip.src', 2);

This rule action would iterate through each ip.src in the initial result set and lookup the top two destination IP addresses with each ip.src.

The following figure shows the rule definition.

Build Rule

Rule Type: **NetWitness Platform DB**

Name: **Lookup and add**

Summarize: **Event Count**

Select: **ip.src**

Alias: **Source IP Address**

Where: **service exists**

Group By: **ip.src**

Then: **lookup_and_add('ip.dst','ip.src',2);**
Enter a then clause...

Order By:

Column Name	Sort By
Total	Ascending

Session Threshold: **2**

Limit: **20**

Use **Save** **Reset** **Test Rule**

The following figure shows the result set containing the source IP addresses and the top two destination IP addresses with each ip.src.

Test Rule

Data Source: **Admin- Concentrator**

Format: **Tabular**

Time Range: **Past**

2 Months

☒ Use relative time calculation

Run Test

2018 01 02 10:12:00		Lookup and add		2018 03 02 10:11:59	
Source IP Address				Total events count	
1. ip.src	192.201.1128.444			1	
1. ip.dst	192.201.1128.444			1	
2. ip.src	192.168.2011.179			1	
1. ip.dst	192.168.2011.179			1	
3. ip.src	192.2716.2018.877			1	
1. ip.dst	192.2716.2018.877			1	
4. ip.src	204.205.1128.444			1	
1. ip.dst	192.205.202.1128			1	
5. ip.src	204.477.448.2011			1	
1. ip.dst	192.205.202.1128			1	
6. ip.src	204.205.202.1128			1	
1. ip.dst	192.205.202.1128			1	
7. ip.src	204.771.448.1128			1	
1. ip.dst	192.205.202.1128			1	
8. ip.src	204.205.1128.444			1	

Close

2a. `lookup_and_add('ip.dst','ip.src', 2); lookup_and_add('service','ip.src', 3);`

This rule action would iterate through each ip.src in the initial result set and lookup the top two destination IP addresses with each ip.src and the top three ports used by each ip.src.

The following figure shows the rule definition.

Build Rule

Rule Type: NetWitness Platform DB

Name: Lookup and add2

Summarize: Event Count

Select: ip.src

Alias: Source IP Address

Where: service exists && ip.src exists

Group By: ip.src

Then:

```
lookup_and_add('ip.dst','ip.src',2);
lookup_and_add('service','ip.dst',2);
```

Enter a then clause...

Order By:

Column Name	Sort By
Total	Descending

Session Threshold: 1

Limit: 20

Use Save Reset Test Rule

The following figure shows the result set containing the source IP addresses and the top two destination IP addresses with each ip.src and the top three ports used by each ip.src.

Test Rule

Data Source: Admin- Concentrator

Format: Tabular

Time Range: Past

2 Months

☒ Use relative time calculation

Run Test

2018 01 02 10:21:00		Lookup and add2		2018 03 02 10:20:59	
Source IP Address	Total events count				
1. ip.src 206.42.199.194	38983				
1. ip.dst 192.168.1.100	27				
1. service OTHER	25				
2. ip.dst 192.168.1.101	26				
1. service OTHER	25				
2. ip.src 192.168.1.102	26810				
1. ip.dst 192.168.1.103	7487				
1. service OTHER	234				
2. service HTTP	191				
2. ip.dst 192.168.1.104	519				
1. service HTTP	57				
2. service OTHER	39				
3. ip.src 192.168.1.105	25325				
1. ip.dst 192.168.1.106	2290				
1. service HTTP	819				

Close

You can make the query as complex as you want by selecting different fields in the result set and by appending to different parts. For example, you may want to know what files each source IP had touched. However, because the parent rule has a WHERE clause of service = 6667 and the default behavior of this rule action is to append to the original WHERE clause, it becomes necessary to override the parent WHERE clause. The easiest way to understand this concept is to look at the previous lookup_and_add call lookup_and_add('ip.dst','ip.src',2). The actual query that is sent to the server is SELECT ip.dst WHERE service = 6667 && ip.src = 206.42.199.194. In order to force the WHERE clause to override the service = 6667 portion of the WHERE clause (inherited from the parent rule), the user can specify a 4th parameter of false as shown in example 3.

2b. Without Lookup_and_add Rule

This rule uses the Custom summarization and Group By feature to sort the results.

The following figure shows the rule definition.

Build Rule

Rule Type: NetWitness Platform DB

Name: Without LUA

Summarize: Custom

Select: ip.src, ip.dst, service, count(sessionid)

Alias: Source IP Address

Where: service exists && ip.src exists

Group By: ip.src, ip.dst, service

Then: Enter a then clause...

Order By:

Column Name	Sort By
count(sessionid)	Descending
Enter the column name...	Ascending

Session Threshold: 0

Limit: 20

Use Save Reset Test Rule

The following figure shows the result set containing the source IP addresses and the top two destination IP addresses with each ip.src and the top three ports used by each ip.src.

Test Rule		2018	01 02	10:41:00	Without LUA		2018	03 02	10:40:59
Data Source	Format	Time Range		Use relative time calculation		Run Test			
Admin- Concentrator	Tabular	Past	2	Months					
		Source IP Address	Destination IP Address	Service Type	count(sessionid)				
1		192.168.1.1	192.168.1.1	SSL	13942				
2		192.168.1.1	192.168.1.1	HTTP	10619				
3		192.168.1.1	192.168.1.1	HTTP	8981				
4		192.168.1.1	192.168.1.1	HTTP	4553				
5		192.168.1.1	192.168.1.1	HTTP	4183				
6		192.168.1.1	192.168.1.1	HTTP	3651				
7		192.168.1.1	192.168.1.1	HTTP	3462				
8		192.168.1.1	192.168.1.1	OTHER	3383				
9		192.168.1.1	192.168.1.1	SSL	2887				
10		192.168.1.1	192.168.1.1	HTTP	2848				
11		192.168.1.1	192.168.1.1	HTTP	2747				
12		192.168.1.1	192.168.1.1	HTTP	2548				
13		192.168.1.1	192.168.1.1	HTTP	2538				
14		192.168.1.1	192.168.1.1	OTHER	2395				
15		192.168.1.1	192.168.1.1	HTTP	2374				
16		192.168.1.1	192.168.1.1	HTTP	2287				

3. lookup_and_add('filename', 'ip.src', 2, false);

This call would issue a query to the server, like SELECT filename WHERE ip.src = 90.0.0.142 rather than SELECT filename WHERE service = 6667' && ip.src = 90.0.0.142 because you have specified the rule action to ignore the initial WHERE clause of the parent rule.

The following figure shows the rule definition.

Build Rule

Rule Type: NetWitness Platform DB

Name: Lookup and add overriding WHERE clause

Summarize: Event Count

Select: ip.src

Alias: Source IP Address

Where: service exists

Group By: ip.src

Then: lookup_and_add('filename','ip.src',2,false);
Enter a then clause...

Order By: Column Name: Total, Sort By: Descending

Session Threshold: 1

Limit: 5

Use Save Reset Test Rule

The following figure shows the result set.

Test Rule

Data Source: Admin- Concentrator

Format: Tabular

Time Range: Past

2 Months

☒ Use relative time calculation

Run Test

2018	01	02	10:48:00	Lookup and add overriding...	2018	03	02	10:47:59
Source IP Address				Total events count				
1. ip.src 128.164.132.33				26810				
1. filename adserver				125				
2. filename + adbrite_iab_iframe_url +				105				
2. ip.src 128.164.75.230				25325				
1. filename online				735				
2. filename bind				698				
3. ip.src 222.88.116.196				24666				
4. ip.src 128.164.141.11				23605				
5. ip.src 166.248.83.83				21495				
1. filename bind				43				
2. filename <none>				22				

Close

The test list is in a group name netwitness, you can access that list with the following syntax.

You can even narrow down these appended results even further to only include filenames that have .gif as filename extension by using the fifth parameter in the rule action. The fifth parameter allows you to specify additional WHERE clause criteria. The files with .gif filename extension would be stored in the **test** list within a group named **DocTeamList**. You can access this list with the following syntax: threat.source = \${DocTeamList/test}

This can be referenced in the extra where clause parameter in the following manner:

4. lookup_and_add('filename', 'ip.src', 5, false, 'filename CONTAINS \${DocTeamList/test}');

The following figure shows the rule definition.

Build Rule

Rule Type:

Name:

Summarize:

Select:

Alias:

Where:

Group By:

Then:
Enter a then clause...

Order By:

Column Name	Sort By
Total	Descending

Session Threshold:

Limit:

The following figure shows the result set.

Test Rule

Data Source:

Format:

Time Range:

☒ Use relative time calculation

2018 02 19 05:11:00		Return non-aggregate valu...	2018 03 05 05:10:59
Source IP address	Total events count		
1. ip.src 192.168.1.1	357293		
1. ip.dst 200.200.200.2			
2. ip.src 200.200.200.2	156871		
1. ip.dst 100.100.100.1			
2. ip.dst 100.100.100.1			
3. ip.src 200.200.200.2	155180		
1. ip.dst 192.168.1.1			
2. ip.dst 200.200.200.2			
4. ip.src 200.200.200.2	64962		
1. ip.dst 192.168.1.1			
2. ip.dst 192.168.1.1			
5. ip.src 100.100.100.1	60124		
1. ip.dst 200.200.200.2			
2. ip.dst 192.168.1.1			
6. ip.src 200.200.200.2	54135		
1. ip.dst 100.100.100.1			

max_threshold (string quantity)

max_threshold (string quantity, string field)

max_threshold removes any results with a quantity that is larger than the maximum threshold quantity from a result set. The quantity can either be in terms of count or size and it is relative to the sorting options of the parent rule. This means that if you sort a rule by size, the rule action expects you to specify the parameter in bytes (you can append KB, MB, GB, TB to the parameter to make size conversion easier).

max_threshold rule can also be used to filter values based on the aggregate function values. Use the syntax based on the type of summarization used in the rule as below:

- max_threshold(String quantity): Can be used to filter Event Count, Packet Count, and Session Size.
- max_threshold(String quantity, String field): Can be used to filter values of Custom aggregates or any metas.

Examples:

1. max_threshold(200);

The following figure shows the result without the max_threshold argument. The output results have event counts exceeding 200.

Test Rule

Data Source: Conc-240

Format: Tabular

Time Range: Past

10 Years

Run Test

SL No	Source IP Address	Total events count
1	192.168.1.100	1884
2	192.168.1.101	6
3	192.168.1.102	6
4	192.168.1.103	6
5	192.168.1.104	6
6	192.168.1.105	6
7	192.168.1.106	6
8	192.168.1.107	6
9	192.168.1.108	6
10	192.168.1.109	6
11	192.168.1.110	6
12	192.168.1.111	6
13	192.168.1.112	6
14	192.168.1.113	6
15	192.168.1.114	6
16	192.168.1.115	6
17	192.168.1.116	6

Close

The following figure shows a the max_threshold rule action that puts a limit of 200 bytes on the output. Any output having more than 200 bytes of data are not listed.

Build Rule

Rule Type: NetWitness Platform DB

Name: Max Threshold

Summarize: Event Count

Select: ip.src

Alias: Source IP address

Where:

Group By: ip.src

Then: Max_Threshold(200);

Order By:

Column Name	Sort By
Total	Descending

Session Threshold: 0

Limit: 20

Use Save Reset Test Rule

The following figure shows the result when the max_threshold rule action is applied. The result numbered 1 in the above screen capture is removed from the result.

Test Rule

Data Source: Conc-240

Format: Tabular

Time Range: Past

10 Years

Run Test

2003	01	01	05:30	Max Threshold	2013	01	01	05:30
SL No	Source IP Address	Total events count						
1	100.100.100.100	6						
2	100.100.100.100	6						
3	100.100.100.100	6						
4	100.100.100.100	6						
5	100.100.100.100	6						
6	100.100.100.100	6						
7	100.100.100.100	6						
8	100.100.100.100	6						
9	100.100.100.100	6						
10	100.100.100.100	6						
11	100.100.100.100	6						
12	100.100.100.100	6						
13	100.100.100.100	6						
14	100.100.100.100	6						
15	100.100.100.100	6						
16	100.100.100.100	6						
17	100.100.100.100	6						

Close

2. max_threshold(5,count(alias.host));

The following figure shows the result without the max_threshold argument. The output results have count of alias.host exceeding 5.

Test Rule

Data Source: 204.31-Conc

Format: Tabular

Time Range: Past

2 Weeks

☒ Use relative time calculation

Run Test

2015	01	25	15:02	Max Threshold Initial	2015	02	08	15:02
	Source IP Address	Source Country	Destination Country	Destination IP address	Source User Account	count (alias.host)		
1	100.100.100.100	United States	United States	100.100.100.100		615		
2	100.100.100.100	United States	United States	100.100.100.100		424		
3	100.100.100.100	United States	United States	100.100.100.100		342		
4	100.100.100.100	United States	United States	100.100.100.100		318		
5	100.100.100.100	United States	United States	100.100.100.100		250		
6	100.100.100.100	United States	United States	100.100.100.100		222		
7	100.100.100.100	United States	United States	100.100.100.100		220		
8	100.100.100.100	United States	United States	100.100.100.100		217		
9	100.100.100.100	United States	United States	100.100.100.100		211		
10	100.100.100.100	United States	United States	100.100.100.100		211		
11	100.100.100.100	United States	United States	100.100.100.100		185		
12	100.100.100.100	United States	United States	100.100.100.100		184		
13	100.100.100.100	United States	United States	100.100.100.100		166		
14	100.100.100.100	United States	United States	100.100.100.100		164		

Close

The following figure shows a the max_threshold rule action that puts a limit of 5 on the output. Any output having value more than 5 is not listed.

Build Rule

Rule Type: NetWitness Platform DB

Name: Max Threshold Count Alias Host

Summarize: Custom

Select: ip.src, country.src, country.dst, ip.dst, user.src, count(alias.host)

Alias: Source IP address

Where: ip.src exists

Group By: ip.src, country.src, country.dst, ip.dst, user.src

Then: Max_Threshold(5, count(alias.host));
Enter a then clause...

Order By:

Column Name	Sort By
count(alias.host)	Descending
Enter the column name...	Ascending

Session Threshold: 0

Limit: 100000

Use Save Reset Test Rule

The following figure shows the result when the max_threshold rule action is applied. Any output having value more than 5 is removed from the result.

Test Rule

Data Source: Admin- Concentrator

Format: Tabular

Time Range: Past

2 Years

☒ Use relative time calculation

Run Test

	2016 03 05 05:31:00	Max Threshold Count Alias...				2018 03 05 05:30:59
	Source IP address	Source Country	Destination Country	Destination IP Address	Source User Account	count(alias.host)
1	192.168.1.100	United States	United States	192.168.1.100		5
2	192.168.1.100	United States	United States	192.168.1.100		5
3	192.168.1.100	United States	United States	192.168.1.100		5
4	192.168.1.100	India	United States	192.168.1.100		5
5	192.168.1.100	United States	United States	192.168.1.100		5
6	192.168.1.100	United States	United States	192.168.1.100		5
7	192.168.1.100	United States	United States	192.168.1.100		5
8	192.168.1.100	United States	United States	192.168.1.100		5
9	192.168.1.100	United States	United States	192.168.1.100		5
10	192.168.1.100	United States	United States	192.168.1.100		5

Close

min_threshold (string quantity)

min_threshold removes results with a quantity that is smaller than the minimum threshold quantity from a result set. The quantity can either be in terms of count or size and it is relative to the sorting options of the parent rule. This means that if you sort a rule by size, the rule action expects you to specify the parameter in bytes (you can append KB, MB, GB, TB to the parameter to make size conversion easier).

min_threshold rule can also be used to filter values based on the aggregate function values. Use the syntax based on the type of summarization used in the rule as below:

- min_threshold(String quantity): Can be used to filter Event Count, Packet Count, and Session Size.
- min_threshold(String quantity, String field): Can be used to filter values of Custom aggregates or any metas.

Examples:

1. min_threshold(200);

The following figure shows a sample of the min_threshold query.

Build Rule

Rule Type: NetWitness Platform DB

Name: Min Threshold

Summarize: Event Count

Select: ip.src

Alias: Source IP address

Where:

Group By: ip.src

Then: Min_Threshold(200);
Enter a then clause...

Order By:

Column Name	Sort By
Total	Ascending

Session Threshold: 0

Limit: 20

Use Save Reset Test Rule

The above figure puts a limit of 200 bytes on the output. Any output having less than 200 bytes of data is not listed. The output with the min_threshold rule action is applied.

Test Rule

Data Source: Conc-240

Format: Tabular

Time Range: Past

10 Years

Run Test

SL No	Source IP Address	Total events count
1	10.10.10.10	1884

Close

As shown, all the values are greater than 200 bytes.

2. min_threshold(100,count(alias.host));

The following figure shows the result without the min_threshold argument. The output results have count of alias.host below 100.

Test Rule

Data Source: 204.31-Conc

Format: Tabular

Time Range: Past

2 Weeks

☒ Use relative time calculation

Run Test

	2015 01 24 16:06	Min Threshold Initial				2015 02 07 16:06
	Source IP Address	Source Country	Destination Country	Destination IP address	Source User Account	count (alias.host)
1	192.168.1.1	United States	United States	192.168.1.1		1
2	192.168.1.1	United States	United States	192.168.1.1		1
3	192.168.1.1	United States	United States	192.168.1.1		1
4	192.168.1.1	United States	United States	192.168.1.1		3
5	192.168.1.1	United States	United States	192.168.1.1		3
6	192.168.1.1	United States	United States	192.168.1.1		4
7	192.168.1.1	United States	United States	192.168.1.1		4
8	192.168.1.1	United States	United States	192.168.1.1		4
9	192.168.1.1	United States	United States	192.168.1.1		4
10	192.168.1.1	United States	United States	192.168.1.1		4
11	192.168.1.1	United States	United States	192.168.1.1		4
12	192.168.1.1	United States	United States	192.168.1.1		4
13	192.168.1.1	United States	United States	192.168.1.1		4
14	192.168.1.1	United States	United States	192.168.1.1		4

Close

The following figure shows a the min_threshold rule action that sets the minimum limit of 100 on the output. Any output having data less than 100 is not listed.

Build Rule

Rule Type: NetWitness Platform DB

Name: Min Threshold Count Alias Host

Summarize: Custom

Select: ip.src, country.src, country.dst, ip.dst, user.src, count(alias.host)

Alias: Source IP address

Where: ip.src exists

Group By: ip.src, country.src, country.dst, ip.dst, user.src

Then: Min_Threshold(5, count(alias.host));

Order By:

Column Name	Sort By
count(alias.host)	Descending
Enter the column name...	Ascending

Session Threshold: 0

Limit: 100000

Use Save Reset Test Rule

The following figure shows the result when the min_threshold rule action is applied. Any output having data less than 100 is removed from the result.

Test Rule

Data Source: Admin- Concentrator

Format: Tabular

Time Range: Past

2 Years

☒ Use relative time calculation

Run Test

	Source IP Address	Source Country	Destination Country	Destination IP Address	Source User Account	count(alias.host)
1	192.168.1.1			192.168.1.1		67886
2	192.168.1.1					28872
3	192.168.1.1					21648
4	192.168.1.1	United States	United States	192.168.1.1		21238
5	192.168.1.1	United States	United States	192.168.1.1		20464
6	192.168.1.1					18045
7	192.168.1.1	United States	United States	192.168.1.1		11664
8	192.168.1.1					10827
9	192.168.1.1					10827
10	192.168.1.1	United States	United States	192.168.1.1		8936
11	192.168.1.1	United States	United States	192.168.1.1		8366
12	192.168.1.1	United States	United States	192.168.1.1		8052
13	192.168.1.1	United States	United States	192.168.1.1		7785
14	192.168.1.1	United States	United States	192.168.1.1		7656

Close

regex (string regex, string field)

The regex rule action applies regular expression to the result set. The following is the format of the regex rule action:

regex(regular_expression, meta_name)

Where:

- regular_expression - Regular expression to match the value of the meta.
- meta_name - Meta or field name on which the regex has to be applied.

For a comprehensive list of supported regex patterns, refer to <http://docs.oracle.com/javase/7/docs/api/java/util/regex/Pattern.html> (<http://docs.oracle.com/javase/7/docs/api/java/util/regex/Pattern.html>).

Sample regex rule action:

If you want to list filenames of all the PNG and JPEG format files from various sessions, you can write a rule with the following regex rule action:

regex(".*.(png|jpg)", filename);

The following figure shows the rule.

Build Rule

Rule Type: NetWitness Platform DB

Name: Reg Expression

Summarize: Event Count

Select: filename

Alias: File Name

Where:

Group By: filename

Then:

regex(".*.(png|jpg)", filename);

Enter a then clause...

Order By:

Column Name: Total

Sort By: Descending

Session Threshold: 0

Limit: 20

Use Save Reset Test Rule

The output with the regex rule action applied is shown in the following figure.

Test Rule

Data Source

Conc-240

Format

Tabular

Time Range

Past

10

Years

Run Test

2003

01

01

05:30

Reg Exp

2013

01

01

05:30

SL No	Filename	Total events count
1	0.jpg	2
2	0000050574_000000000000000546126.jpg	2
3	01-28-2008_18month3no_widget.jpg	2
4	01010901030801160220080213fabfe407e7f75bb543004d28.jpg	2
5	01021101030101161020080212a935b5807a3f8069de001897.jpg	2
6	01440gk04el.jpg	2

Close

sum_count()

Totals the quantifiers for a given result set. For example, calling a sum_count() for a rule that is sorted by event count totals the size of all values in the result set and displays the total in place of the result set.

Example:

The following figure shows the sum_count() rule action.

Build Rule

NetWitness Platform DB

Name	Sum fields							
Summarize	Event Count	▼						
Select	country.src							
Alias	Sum							
Where	<u>country.src</u> = 'United States','China','Canada'							
Group By	country.src							
Then	sum_count(); Enter a then clause... ◀ ▶							
Order By	<table><thead><tr><th>Column Name</th><th>Sort By</th></tr></thead><tbody><tr><td>Total</td><td>Descending</td></tr><tr><td colspan="2"></td></tr></tbody></table>		Column Name	Sort By	Total	Descending		
Column Name	Sort By							
Total	Descending							
Session Threshold	0	↕						
Limit	20	↕						
Use Save Reset Test Rule								

With sum_count() rule action, the output shows the total size of all the event counts.

Test Rule

Data Source: Admin- Concentrator

Format: Tabular

Time Range: Past

2 Years

☒ Use relative time calculation

Run Test

2016 03 05 05:50:00		Sum fields		2018 03 05 05:49:59	
		Sum			Total events count
1	Total Session_count of country.src			2330415	

Close

sum_values()

Totals the number of values for a given result set. Use this action to display how many matches exists for a given rule.

Example:

The following figure shows the sum_values() rule action.

Build Rule

Rule Type: NetWitness Platform DB

Name: Sum Values

Summarize: Event Count

Select: country.src

Alias: Values

Where:

Group By: country.src

Then:

sum_values();

Enter a then clause...

Order By:

Column Name	Sort By
Total	Descending

Session Threshold: 1

Limit:

Use Save Reset Test Rule

The following figure shows the result with sum_values rule action.

Test Rule

Data Source: Admin- Concentrator

Format: Tabular

Time Range: Past

2 Years

☒ Use relative time calculation

Run Test

2016	03 05	05:54:00	Sum Values	2018	03 05	05:53:59
No of unique country.src values						
1	178					

Close

show_whats_new()

The show_whats_new() rule action takes any result in a result set and filters out any value that is available in the NetWitness meta database prior to the time frame of the currently running report. When a report runs, NetWitness determines the ID of the first session in the time range of the report. If a value in a result set has a first session id that is greater than the first session id of the report time frame, it did not exist in the NetWitness meta database prior to the report being run and so is new to the NetWitness system relative to the time frame of the report.

The show_whats_new() rule action is also supported for Custom Aggregate Rule. When multiple meta's are selected in the Custom rule, the first meta is considered for filtering out the old values. See "Example 2" below to understand how this rule action is used for Custom Aggregate Rule.

Note: The show_whats_new() rule action can be used only with an aggregate rule.

Examples:

1. show_whats_new() for aggregate rule with Event Count

In the following example, all the Source IP Addresses available for the past two weeks are listed.

Test Rule

Data Source: 204.31-Conc

Format: Tabular

Time Range: Past

2 Weeks

☒ Use relative time calculation

Run Test

2015	01 27	12:12:59	WO_SWN	2015	02 10	12:12:59
		Source IP Address				Total events count
1	192.168.1.1					58594
2	192.168.1.2					12073
3	204.31.20.2					5048
4	204.31.20.3					2298
5	192.168.1.3					2238
6	192.168.1.4					1770
7	192.168.1.5					1709
8	192.168.1.6					1684
9	192.168.1.7					1437
10	192.168.1.8					1408
11	192.168.1.9					1112
12	192.168.1.10					905
13	192.168.1.11					899
14	192.168.1.12					822
15	192.168.1.13					812

Close

The following figure shows the use of the show_whats_new rule action to list only the new entries for the past two weeks.

Build Rule

Rule Type:

Name:

Summarize:

Select:

Alias:

Where:

Group By:

Then:

show_whats_new();

Enter a then clause...

Order By:

Column Name	Sort By
Total	Descending

Session Threshold:

Limit:

The following figure lists the new entries for the past two weeks.

Test Rule

Data Source:

Format:

Time Range:

☒ Use relative time calculation

2018	02	19	05:56:00	ShowWhatsNew	2018	03	05	05:55:59
				Values	Total events count			
1								26810
2								25325
3								23605
4								21495
5								11928
6								6750
7								6671
8								6541
9								6086
10								6010
11								5820
12								5760
13								5692
14								5606
15								5329
16								4621

2. show_whats_new() for Custom aggregate rule

In the following example, all the Source IP Addresses available for the past two weeks are listed.

Test Rule

Data Source
204.31-Conc

Format
Tabular

Time Range
Past

2 Weeks

☒ Use relative time calculation

Run Test

	2015 01 27 12:27:35	WO_SWN_aggregate	2015 02 10 12:27:35
	Source IP Address	sum(size)	
1	2001:2001:2001:2001	51416	
2	2001:2001:2001:2001	5760	
3	2001:2001:2001:2001	16936	
4	2001:2001:2001:2001	3952	
5	2001:2001:2001:2001	67430	
6	2001:2001:2001:2001	3920	
7	2001:2001:2001:2001	16956	
8	2001:2001:2001:2001	17898	
9	2001:2001:2001:2001	3696	
10	2001:2001:2001:2001	11520	
11	2001:2001:2001:2001	18277636	
12	2001:2001:2001:2001	2048	
13	2001:2001:2001:2001	62340	
14	2001:2001:2001:2001	13374	
15	2001:2001:2001:2001	5472	

Close

The following figure shows the use of the show_whats_new rule action to list only the new entries for the past two weeks.

Build Rule

Rule Type
NetWitness Platform DB

Name
SWN_aggregate

Summarize
Custom

Select
ip.src, sum(size)

Alias
Source IP Address

Where
ip.src exists

Group By
ip.src

Then
show_whats_new();
Enter a then clause...

Order By

Column Name	Sort By
ip.src	Descending
Enter the column name...	Ascending

Session Threshold
0

Limit
2000

Use **Save** **Reset** **Test Rule**

The following figure lists the new entries of Source IP Addresses for the past two weeks.

Data Source

10.31.126.151 - Concentra

Format

Tabular

Time Range

Past

2

Days

Use relative time calculation

Run Test

2015	02	08	10:41	ShowWhatsNew	2015	02	10	10:41
				Source IP Address				sum(size)
1				200.217.128.86				1788
2				200.186.186.186				1788
3				200.126.86.87				1632
4				200.86.86.186				1788
5				200.87.128.86				261084
6				200.86.86.186				1764
7				200.86.86.186				596
8				200.86.248.86				166284
9				200.86.208.112				1764
10				200.200.128.186				57904
11				200.200.128.207				149436
12				200.216.86.208				398568
13				200.200.208.187				4176
14				200.186.128.186				1764
15				200.128.186.186				1764

Close

The power of this feature is that it doesn't matter when the report is run in identifying values that are new to NetWitness. The caveat with this feature is that if a data reset occurs, you will lose your data. However, it is easy to baseline a system and identify changes and new items without a tremendous amount of strain on the system (depending on the size of your result set).

Supported Rule Operators
The NWDB Reporting Engine data source rule syntax supports a subset of rule operators that are supported by NetWitness.

Syntax	Description
*	Use an asterisk (*) as the sole operator in a rule to select all traffic.
=	Equals operator
!=	Does not equal operator
&&	Logical AND operator
	Logical OR operator
-u	Upper boundary. For example, tcp.port = 40000-u selects all TCP ports above 40000.
l-	Lower boundary. For example, tcp.port = l-40000 selects all TCP ports below 40000.
-	The dash (-) operator only applies to numeric values. Separate the lower and upper boundaries of the range with a dash (-). For example, tcp.port = 25-443 selects all TCP ports between 25 and 443.

IPDB Rule SyntaxIPDB Rule Syntax

Sample Supported QueriesSample Supported Queries

Respond Rule SyntaxRespond Rule Syntax

The supported rule syntax for the Respond service through descriptions and examples of supported and unsupported syntax. There is a finite set of syntax that you can use to construct rules for reports using the Respond service in this release.

The Reporting Engine supports the following categories of Respond data source rule syntax:

- **select** clause
 - Non-Aggregate Rule
 - Aggregate Rule
- **alias**
- **where** clause
- **where** clause Operators
- Group By
- Order By
- **Limit** field

Note: List is not supported in Respond Data source rules.

Select Clause

The select clause is a comma separated list of values. For example: select alert.severity, alert.name (https://alert.name), count(*).

There are two types of select clause for Respond Rule:

- Non-aggregate rule

- Aggregate rule

Non-Aggregate Rule

When you want to define a rule without any grouping, choose 'None' in the Summarize field. In a non-aggregate rule, you can select any number of metas in the *Select* clause. For example, select alert.severity, [alert.name \(https://alert.name\)](https://alert.name).

Aggregate Rule

When you want to query for a specific meta and its associated aggregate value then you must use the Aggregate rule. To get an aggregate, you must choose 'Custom' in the Summarize field to include an aggregate function in the *Select* clause. For example, select alert.severity, [alert.name \(https://alert.name\)](https://alert.name), count(*).

The following figure shows the Build Rule view for Aggregate Rule.

Build Rule

Rule Type: Respond DB

Name: TestScreenshot

Summarize: None

From: alert

Select: alert.name, alert.severity, count(alert.name)

Alias: Name

Where: alert.name="Brute Force Login From Same Source"

Order By:

Column Name	Sort By
Enter the column name...	Ascending

Limit: 20

Buttons: Use, Save, Reset, Test Rule

Supported Aggregate Functions

The rules on Respond service supports the following aggregate functions and syntax.

- count
- max
- min
- sum
- avg

Note: The aggregate functions must be added in the end of a select clause for aggregate query. For example, [alert.name \(https://alert.name\)](https://alert.name), alert.severity, sum(alert.numEvents). By default, a maximum of 10,000 rows results are fetched and this can be configured using the `rsa.response.query.QueryProperties`.

Examples of select Clause Syntax

The following table provides examples of the select Clause Syntax.

Examples	Descriptions
select column1,column2,column3,...,columnN	Select specific metas from an Respond Data Source (You must separate each column with a comma.).

Examples of Supported Select Queries

```
select alert.name \(https://alert.name\), alert.numEvents, count(alert.numEvents)
select alert.severity, avg(alert.severity)
select alert.timestamp, incidentCreated where alert.timestamp >= 1475658011
```

Summarize

Summarize determines the type of summarization or aggregation for the rule.

Name	Config Value
	To query metas without any custom grouping, select: None:
Summarize	To get meta based aggregates, select: Custom: This indicates that expected meta aggregate function is defined in rule select clause.

Alias

Some meta names may not be descriptive, in this case description can be added in the the alias field to make column names more readable. For example, **SELECT:** alert.severity, [alert.name \(https://alert.name\)](https://alert.name), count(*)

ALIAS: Alert Severity, Alert Name

In the alias field you can enter a name for columns used in the select clause. If you do not specify the alias for one of the field in the select clause, then the default description will be used. For example, if the select clause has Field1,Field2,Field3,Field4, and alias has only Field1, ,Field3,Field4, then for Field2 a default description is used.

Where Clause

The where clause is a language field values and ranges that is used by Respond function. In the where clause, string values have to be enclosed within single quotes.

Examples	Descriptions
alert.host summary ='(Primary) Link status "Down" on interface INTNAME'	For TEXT or string type data, enclose the string or text in single or double quote. If there is any special character such as an apostrophe within the data then you need to add an additional single or double quotes. For example, <u>alert.name (https://alert.name)</u> = 'top alerts from Cote d'Ivoire'.
alert.timestamp >= 1475658011	For Date and Time (date/timestamp data type columns), use the EPOCH syntax.

Supported Where Clause Operators

Operator	Syntax
= (equals)	<i>column1</i> = 'value'
!= (does not equal)	<i>column1</i> != 'value'
>	<i>column1</i> > 'value'
>=	<i>column1</i> >= 'value'
<	<i>column1</i> < 'value'
<=	<i>column1</i> <= 'value'

Group By

Syntax	Function
group by : alert.severity, alert.timestamp, incidentCreated Note: Group by field is enabled for Aggregate queries and are not editable.	Respond picks the metas for Group By field from the selected Select clause automatically.

Order By

Order By determines how to sort the result set and is not case sensitive.

Name	Configuration Value
Column Name	The Column Name is the name of the columns by which you want to sort the results. By default, the value is empty. When you click on a column, the value gets populated based on the Summarize field. - order by <u>alert.name (https://alert.name)</u> asc - order by incidentCreated desc - order by count(numEvents) - order by status
Sort By	Sort By determines the order in which you want to sort the results such as ascending or descending. Note: For all queries, it is mandatory for you to select the order by field.
Limit field	
This indicates the limit to be put on the query while fetching data from the database. If a result set is sorted by event count, packet count, or session size, the limit represents the top (or bottom) N values to be returned. If the result set is not sorted, the first N values are returned.	

[Previous Page \(https://community.netwitness.com/s/article/670015\)](https://community.netwitness.com/s/article/670015) [Next Page \(https://community.netwitness.com/s/article/WarehouseDBSimpleRules\)](https://community.netwitness.com/s/article/WarehouseDBSimpleRules)

FOLLOW

RELATED ARTICLES

Rule Syntax Dialog (/s/article/RuleSyntaxDialog)	1
Understanding and Creating RSA NetWitness Endpoint Alerts in v11.3 (/s/article/UnderstandingandCreatingRSANetWitnessEndpointAlertsinv11-3)	3

Fix Rules with Invalid Syntax (/s/article/FixRuleswithInvalidSyntax)

0

ESA Rule Syntax Error when using the meta "index" to lowercase (/s/article/ESA-Rule-Syntax-Error-when-using-the-meta-index-to-lowercase)

136

How to use the "not begins" operator in Netwitness Reporting Engine query (/s/article/How-to-use-the-not-begins-operator-in-Netwitness-Reporting-Engine-query)

18

Sort by:

Latest Posts ▾

Search this feed...

▼

↺



Nothing here yet?

Log in to post to this feed.

[NetWitness Platform](https://www.netwitness.com/)

(<https://www.netwitness.com/>)

[Acceptable Use Policy](https://www.netwitness.com/wp-content/uploads/NetWitness-Acceptable-Use-Policy-092022.pdf)

(<https://www.netwitness.com/wp-content/uploads/NetWitness-Acceptable-Use-Policy-092022.pdf>)

[Privacy Statement](https://www.netwitness.com/wp-content/uploads/NetWitness-Privacy-Statement-07292024.pdf)

(<https://www.netwitness.com/wp-content/uploads/NetWitness-Privacy-Statement-07292024.pdf>)



© 2025 NetWitness LLC. All rights reserved.
(<https://www.linkedin.com/company/netwitness>)
(<https://twitter.com/NetWitness>)